KARNAVATI UNIVERSITY
UNITEDWORLD INSTITUTE OF TECHNOLOGY (UIT)

MINI PROJECT REPORT

ON

"AeroMind: Multi-Agent Post-Flight Analysis of ArduPilot Drone Missions"

Submitted as a part of the course

AGENTIC AI

Submitted By

Student Name	Enrollment No.
_____ _	_____

Program: B.Tech CSE / B.Tech CSE-AIML
Semester: VI
Section: _____

Under the Guidance of

Faculty Name
Assistant Professor
Department of Computer Science & Information Technology

Karnavati University, Gandhinagar

Academic Year: 2026–27

CERTIFICATE

This is to certify that the mini project entitled **“AeroMind: Multi-Agent Post-Flight Analysis of ArduPilot Drone Missions”** has been successfully completed by the following students of B.Tech AeroMind: Multi-Agent Post-Flight Analysis of ArduPilot Drone Missions Semester VI, Karnavati University, as a part of the course **Agentic AI** during the academic year 2026–27.

Sr. No.	Student Name	Enrollment No.
1		
2		
3		
4		

The project has been carried out under the guidance and supervision of the faculty member. The work presented in this report is the original work carried out by the students as part of their mini project.

Faculty
Name:
Signature:
Date: _____

Guide
_____
_____

DECLARATION

We hereby declare that the mini project entitled **"AeroMind: Multi-Agent Post-Flight Analysis of ArduPilot Drone Missions"** submitted as part of the course **Agentic AI** is our original work carried out under the guidance of the faculty member.

We further declare that this work has not been submitted previously to any other institution or university for the award of any degree, diploma, or academic certification.

All research papers, datasets, frameworks, APIs, libraries, websites, and other resources used during the development of the project have been appropriately acknowledged and referenced.

Sr. No.	Student Name	Enrollment No.
1		
2		
3		
4		

Date: _____

Place: Gandhinagar

ACKNOWLEDGEMENT

We thank our faculty guide for the direction, feedback and patience given throughout this mini project, and the Department of Computer Science & Information Technology, Unitedworld Institute of Technology, Karnavati University, for including Agentic AI in the curriculum and for the laboratory and computing facilities that made the work possible.

The simulation work builds on the disaster-zone world and the Hexa-X airframe our team developed for the Smart India Hackathon 2026 (the Aero Sense project), and on open-source software from the ArduPilot, Gazebo and Python communities: ArduPilot SITL, MAVProxy and pymavlink, Gazebo Harmonic, Streamlit, Plotly, pandas, Pydantic and Ollama. We acknowledge the authors of the third-party simulation models listed in the references.

Finally, we thank every team member for their contribution to the design, implementation, testing and documentation of AeroMind.

ABSTRACT

After a drone flight, the information needed to understand what happened is scattered across a telemetry log, a mission file, a parameter file and the pilot's memory, and reading it takes expertise. Most operators therefore never learn why a flight drifted, ran its battery low or lost its telemetry link, and repeat the same mistakes. This project presents **AeroMind**, a multi-agent Agentic AI system that analyses the files of one ArduPilot flight and produces a one-page verdict, a phase-by-phase story, a telemetry-link timeline, root causes, a next-mission risk score and a corrected mission file.

Deterministic Python tools convert MAVLink logs into verified metrics with Good/Caution/Bad statuses. An orchestrator builds a manifest, obtains a plan from a large language model (LLM), checks the plan against rules, and runs five specialist agents (Flight Summary, Telemetry Link, Diagnosis, Risk and Advisor) on a shared state. A deterministic Verifier rejects any claim not supported by the data and triggers one self-correction round. A flight history provides long-term memory: personal baselines, battery prediction and recurring issues. The system is built in Python with Pydantic, Streamlit and Plotly, and runs with Claude through the Anthropic API or a local Qwen2.5-3B model through Ollama.

Eight flights were flown in ArduPilot SITL in a Gazebo disaster world with injected faults and known ground truth. AeroMind's agents detected all four injected faults with the correct root cause, located both telemetry dropouts within 0.1 s of the truth, raised no false alarm on the baseline flight and ranked next-mission risk correctly. The mission they corrected for the undersized-battery scenario raised the battery at landing from 14 % to 25 %, and the speed fix reduced the wind-scenario cross-track error from 1.8 m to 1.44 m. The work shows that LLM agents paired with deterministic verification produce useful, grounded flight analysis even with a small model running locally.

TABLE OF CONTENTS

ACKNOWLEDGEMENT....4
ABSTRACT....5
LIST OF FIGURES....8
LIST OF TABLES....9
CHAPTER 1 – INTRODUCTION....10
1.1 Introduction....10
1.2 Background....10
1.3 Problem Statement....10
1.4 Motivation....11
1.5 Objectives....11
1.6 Scope of the Project....11
CHAPTER 2 – LITERATURE REVIEW....12
2.1 Existing Systems and Research....12
2.2 Comparative Analysis....12
2.3 Research Gap....12
CHAPTER 3 – PROPOSED SYSTEM....13
3.1 Proposed Solution....13
3.2 System Requirements....13
3.3 Functional Requirements....14
3.4 Non-Functional Requirements....14
CHAPTER 4 – AGENT ARCHITECTURE....15
4.1 Overall System Architecture....15
4.2 Architecture Components....15
4.3 Agent Roles and Responsibilities....16
4.4 Agent Workflow....16
CHAPTER 5 – AGENTIC AI CONCEPTS USED....17
5.1 ReAct....17
5.2 Planning....17
5.3 Tool Calling....17
5.4 Memory....18
5.5 Retrieval-Augmented Generation....18
5.6 Multi-Agent System....18
5.7 Human-in-the-Loop....19
CHAPTER 6 – METHODOLOGY....20
6.1 Input Processing....20
6.2 Task Understanding....20
6.3 Task Planning....20
6.4 Reasoning and Decision Making....20

6.5 Tool Selection....21
6.6 Task Execution....21
6.7 Observation and Feedback....21
6.8 Final Response Generation....22
6.9 Evaluation Environment: Simulated Flights with Known Faults....22
CHAPTER 7 – IMPLEMENTATION....24
7.1 Project Structure....24
7.2 Agent Implementation....24
7.3 Tool Implementation....24
7.4 Memory Implementation....24
7.5 RAG Implementation....24
7.6 User Interface....25
CHAPTER 8 – RESULTS AND OUTPUT....27
8.1 Test Cases....27
8.2 Sample Outputs....27
8.3 Agent Execution Trace....28
CHAPTER 9 – PERFORMANCE EVALUATION....29
CHAPTER 10 – CHALLENGES AND LIMITATIONS....31
CHAPTER 11 – FUTURE SCOPE....32
CHAPTER 12 – CONCLUSION....33
REFERENCES....34
APPENDIX....36
Appendix A – Source Code....36
Appendix B – Additional Screenshots....36
Appendix C – Sample Prompts....40
Appendix D – Test Data....41

LIST OF FIGURES

Figure 1: Overall system architecture of AeroMind.....15
Figure 2: Agent workflow of one analysis.....16
Figure 3: Planning and execution flow of the orchestrator.....17
Figure 4: The Gazebo disaster world used for the evaluation flights.....23
Figure 5: The Hexa-X hexacopter during a SITL mission.....23
Figure 6: Home page: input form and the Summary tab (S4).....25
Figure 7: Agents tab (S3): plan, Verifier verdicts and each agent's execution.....26
Figure 8: Ask tab (S3): an answer verified against the cited metrics.....26
Figure 9: Generated report.html for S2.....28
Figure 10: Track tab (S3): planned route, flown path coloured by link quality.....36
Figure 11: Telemetry tab (S3): link strip, message rate and dropout table.....37
Figure 12: History tab (S2 fixed): baseline, trends and what worked before.....38
Figure 13: Power & attitude tab (S2).....39
Figure 14: Hexa-X over the collapsed buildings.....40
Figure 15: Hexa-X flying past the flood zone.....40

LIST OF TABLES

Table 1: Literature review summary.....12
Table 2: Comparison of existing tools and the proposed system.....12
Table 3: Hardware requirements.....13
Table 4: Software requirements and technologies used.....13
Table 5: Functional requirements.....14
Table 6: Architecture components.....15
Table 7: Agent roles and responsibilities.....16
Table 8: Tools: purpose, input and output.....18
Table 9: Main metrics and thresholds.....20
Table 10: Base risk rules (Low < 30, Medium 30–59, High ≥ 60).....21
Table 11: Mission edit actions and limits.....21
Table 12: Evaluation scenarios.....22
Table 13: Test cases and actual results.....27
Table 14: Success metrics (computed by run.py --results).....29
Table 15: Performance evaluation.....29
Table 16: Results before and after the fixed missions.....30
Table 17: Sample prompts.....41
Table 18: Test data.....41

CHAPTER 1 – INTRODUCTION

1.1 Introduction

Artificial Intelligence (AI) builds systems that perceive, reason and decide. **Generative AI** produces new content: large language models (LLMs) such as Claude and Qwen generate text, code or structured data from a prompt. **Agentic AI** uses an LLM as the decision-making core of a system that pursues a goal over several steps: it plans, acts through tools or other agents, observes the results and adjusts [1], [11]. A chatbot answers one message at a time; an agent works towards a goal. AeroMind shows each property of an agent:

- **Autonomy:** once a flight's files are dropped in, the whole analysis runs without further input.
- **Planning:** an orchestrator decides which agents to run, in what order and why.
- **Reasoning:** each agent explains what happened and why, citing the metrics it relies on.
- **Tool usage:** Python tools parse the logs, compute metrics, predict battery use and edit the mission.
- **Memory:** every analysed flight is stored and later flights are compared with the pilot's own baseline.
- **Goal-oriented execution:** work continues, with a self-correction round, until the result is verified.

1.2 Background

Drones running the open-source ArduPilot autopilot record their state as MAVLink telemetry, saved by ground stations such as Mission Planner as a `.tlog` file [8], [9]. Existing tools such as ArduPilot's UAV Log Viewer [10] and PX4 Flight Review [12] plot this data well but leave interpretation to the user, who must know which message holds the cross-track error, what is normal and how to turn it into a better mission. AeroMind gives the arithmetic to tools and the judgement to agents, and checks the agents against the tools.

1.3 Problem Statement

The objective of this project is to develop an Agentic AI-based system that analyses the telemetry log, mission and parameters of an ArduPilot flight, explains how the flight and its

telemetry link went, identifies what went wrong and why, scores the risk of the next mission and produces a corrected mission file. Existing systems plot the data but leave diagnosis, risk assessment and mission correction to an expert. Therefore, there is a need for a system that can reason over flight data like an expert while every statement it makes is checked against the data.

1.4 Motivation

Our team builds drones for the Smart India Hackathon and spent hours working out from logs why a flight behaved as it did; small teams rarely have a flight-test engineer. Plotting tools leave interpretation to the user, and a stand-alone LLM can state numbers that are not in the data [6]. Agentic AI lets planned, specialised agents explain and advise while tools keep every number exact.

1.5 Objectives

1. To design a multi-agent system that turns one flight's files into a verdict, a flight story, a telemetry-link analysis, root causes, a next-mission risk score and a corrected mission.
2. To implement LLM-planned orchestration of specialist agents over a shared state, with rule-checked plans.
3. To integrate deterministic tools for log ingestion, metrics, risk scoring and bounded mission editing.
4. To implement long-term memory of flights for baselines, battery prediction and recurring issues.
5. To verify every agent claim against the data and evaluate the system on flights with known injected faults.

1.6 Scope of the Project

- **Target users:** pilots and student teams flying ArduPilot multirotors.
- **Supported tasks:** post-flight analysis of `.tlog` files (several per flight are merged) with an optional mission, parameters and pilot notes; questions about the flight; a corrected mission file.
- **Boundaries:** post-flight analysis, evaluated on simulated flights; by design the system never uploads anything to the drone.

CHAPTER 2 – LITERATURE REVIEW

2.1 Existing Systems and Research

Table 1 summarises the work that shaped AeroMind.

Table 1: Literature review summary

Sr. No.	Paper/System	Year	Approach	Key Finding	Limitation
1	ReAct [1]	2023	Reasoning traces with tool actions	Reasoning and acting beat either alone	Errors propagate
2	AutoGen [2]	2023	Conversing specialist agents	Tasks decompose well across agents	No check against data
3	MetaGPT [3]	2024	Role agents, structured hand-offs	Structure reduces errors	Software engineering only
4	Reflexion [4], Self-Refine [5]	2023	Self-critique and retry	Feedback improves results	Critic can accept wrong answers
5	Log viewers [9], [10], [12]	–	Plot flight logs	Reliable plots of every signal	Interpretation left to user

2.2 Comparative Analysis

Table 2: Comparison of existing tools and the proposed system

Feature	Existing System (log viewers)	Proposed System (AeroMind)
Autonomous Execution	No: the user reads each plot	Yes: files in, full analysis out
Task Planning	No	Orchestrator plans the agents; plan checked by rules
Reasoning	No	Agents explain causes and risk, citing metrics
Tool Usage	Plotting only	Ingest, metrics, memory, risk rules, mission editing
Memory	No	Flight history: baselines, battery prediction, recurring issues
RAG	No	Not used
Multi-Agent Support	No	Five specialist agents and a Verifier
Human Intervention	Entirely manual	Pilot reviews the fixed mission before flying

2.3 Research Gap

As Table 2 shows, log tools are exact but explain nothing, while agent frameworks [1]–[5] do not check what an agent says against data [6]. AeroMind combines both with a Verifier.

CHAPTER 3 – PROPOSED SYSTEM

3.1 Proposed Solution

The user uploads a flight's files in a Streamlit dashboard (or uses the `run.py` command line) and may add pilot notes. Python tools turn the files into metrics on a shared state; long-term memory adds baselines and a battery prediction. The orchestrator plans which of five agents to run (Flight Summary, Telemetry Link, Diagnosis, Risk, Advisor); each is a system prompt and a Pydantic output schema, executed on Claude through the Anthropic API [14] or a local Qwen2.5-3B model [13] through Ollama [15]. The Verifier checks the agents' output, an agent with too many rejected claims refines its answer once, and the approved result is shown in the dashboard and written as `report.html`, `mission_fixed.waypoints` and `changelog.md`. No document retrieval (RAG) is used.

3.2 System Requirements

3.2.1 Hardware Requirements

Table 3: Hardware requirements

Component	Requirement (as used)
Processor	AMD Ryzen 5 7535HS, 6 cores / 12 threads
RAM	16 GB (analysis alone needs under 1 GB)
Storage	About 25 MB for the project and flights; 3.3 GB for the optional local LLM
GPU	NVIDIA GeForce RTX 2050, 4 GB (optional: Gazebo rendering and faster local LLM inference)
Internet Connection	Optional: for the Claude API and map tiles

3.2.2 Software Requirements

Table 3 lists the hardware used and Table 4 the software actually used.

Table 4: Software requirements and technologies used

Software/Technology	Version/Specification
Operating System	Ubuntu 22.04.5 LTS
Programming Language	Python 3.10.12 (Lua for the simulated battery)
IDE/Environment	Visual Studio Code; Git; pytest
Agent Framework	Own orchestrator in Python with Pydantic 2.13 schemas
LLM	Claude (Anthropic SDK 2.1.0) or Qwen2.5-3B via Ollama 0.40.8
Vector Database / Embeddings	Not used
Frontend	Streamlit 1.57, Plotly 6.7; Jinja2 3.1 for report.html
Data and parsing	pymavlink 2.4.49, pandas 2.3, NumPy 2.2
Simulation	ArduCopter SITL (ArduPilot 4.6 development build), MAVProxy 1.8.74, Gazebo Harmonic 8.15

3.3 Functional Requirements

Table 5 lists what the system does; the test cases in Chapter 8 exercise these requirements.

Table 5: Functional requirements

ID	The system shall…
FR1	Accept one or more .tlog files, an optional mission, parameters and pilot notes
FR2	Compute flight, battery, tracking, vibration, GPS, wind, attitude and link metrics with statuses
FR3	Detect telemetry dropouts with time, duration, position and nearest waypoint
FR4	Plan which agents to run and run them on a shared state
FR5	Explain the flight, diagnose causes, score next-mission risk and recommend improvements
FR6	Verify every agent claim and let an agent refine a rejected answer
FR7	Remember flights for baselines, battery prediction and recurring issues
FR8	Apply mission edits within hard limits and write a corrected mission
FR9	Answer questions about the flight from verified data

3.4 Non-Functional Requirements

- **Accuracy:** no number reaches the user unchecked; thresholds follow ArduPilot guidance.
- **Reliability:** the analysis always completes, even if an LLM call does not succeed.
- **Performance:** the tool stage takes under 1 s per flight; total time is set by LLM inference.
- **Security:** validated uploads, escaped display text, localhost-only dashboard, keys in an untracked `.env`.
- **Usability and maintainability:** one-page summary; small typed modules; 247 automated tests.

CHAPTER 4 – AGENT ARCHITECTURE

4.1 Overall System Architecture

Figure 1 shows the architecture as implemented: inputs pass through Python tools into a shared state, the orchestrator plans and runs the agents, the Verifier filters their outputs and only the approved set reaches the outputs and the flight history.

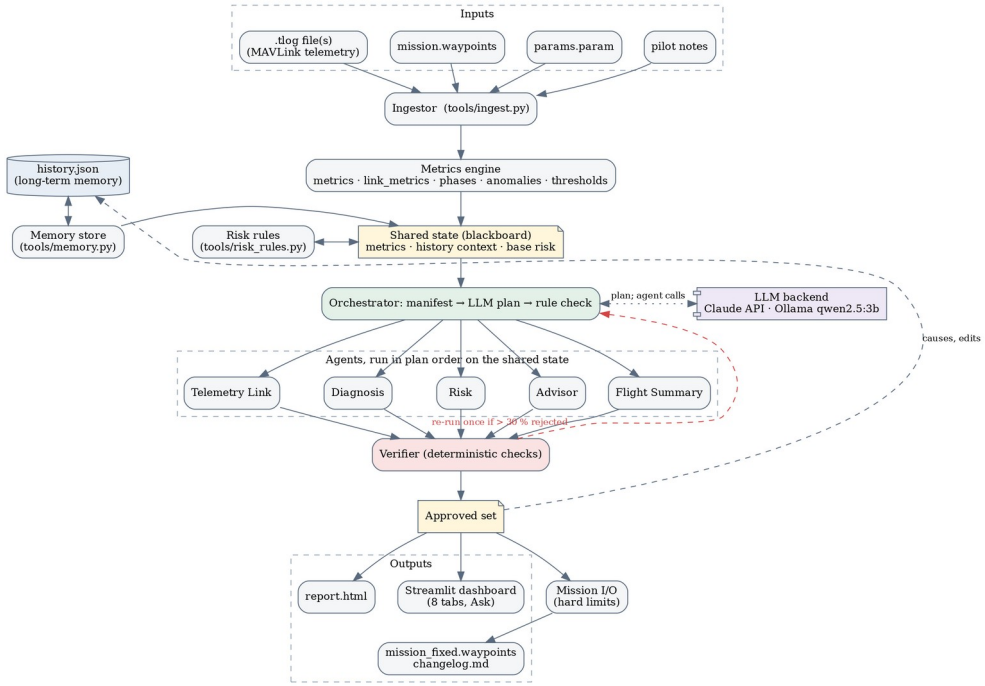


Figure 1: Overall system architecture of AeroMind

4.2 Architecture Components

Table 6: Architecture components

Component	Implementation
User Interface	Streamlit dashboard `app.py` (8 tabs, Ask chat); command line `run.py`
LLM	`agents/llm.py`: Claude (structured output) or Ollama (JSON schema)
Agents	Five specialist agents: prompt (`prompts.py`) and schema (`schemas.py`), run by `run_agent()`
Planner / Executor	`orchestrator.py`: manifest → LLM plan → rule check → run → verify → approve
Tools	`tools/`: ingest, metrics, link metrics, phases, anomalies, risk rules, memory, mission I/O
Memory	`history.json` (long-term) and the shared state of one analysis (working memory)
Output Generator	`approve()` and `pipeline.py`: state.json, report.html, fixed mission, change log

Table 6 maps each component to its implementation.

4.3 Agent Roles and Responsibilities

Table 7: Agent roles and responsibilities

Agent	Role	Responsibility
Orchestrator	Planner, executor	Plans, checks the plan, runs agents, triggers refinement
Flight Summary	Narrator	Verdict, a line per flight phase, 3-sentence summary
Telemetry Link	Link analyst	Uptime, dropouts (time, duration, waypoint), packet loss
Diagnosis	Analyst	Findings citing metrics; up to 3 root causes with confidence
Risk	Assessor	Adjusts the rule-based risk score by at most ±10 with a reason
Advisor	Mission planner	Mission edits tied to causes; top 3 improvements
Verifier	Reviewer	Checks every claim and number against the data

Table 7 lists the agents actually implemented.

4.4 Agent Workflow

Figure 2 shows one analysis: the user provides the files; tools compute metrics and memory; the orchestrator plans; agents run in order; the Verifier observes their outputs; an agent with more than 30 % of its claims rejected refines its answer once; the approved result is saved and presented.

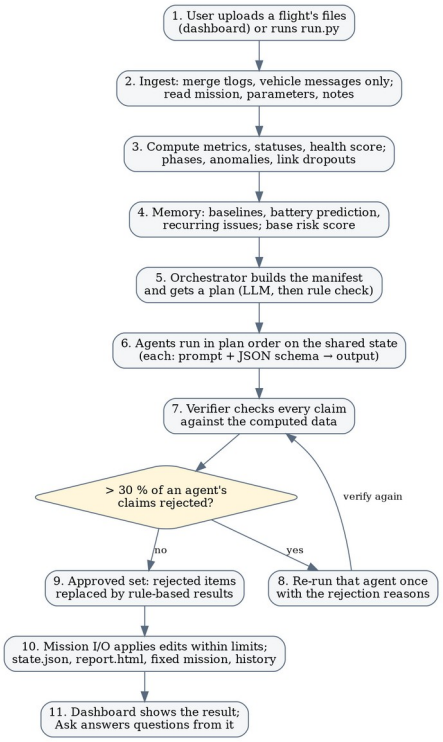


Figure 2: Agent workflow of one analysis

CHAPTER 5 – AGENTIC AI CONCEPTS USED

5.1 ReAct

AeroMind does not use the ReAct loop [1] in which the LLM picks its own tools. The tools always run first, in a fixed order, so no tool can be skipped and the LLM never reads raw telemetry. The reason → act → observe cycle appears instead as verify and refine: the Verifier observes each agent's output and the agent acts again with the feedback.

5.2 Planning

The orchestrator uses plan-and-execute (Figure 3). The LLM receives a manifest (inputs present, metric statuses, problems, link health, flights in history) and returns an ordered list of agents with a reason each. `checked_plan()` then enforces the rules: Flight Summary first, Verifier last, Diagnosis only when something is Caution or Bad, required agents present, dependency order. The plan is dynamic: a healthy flight skips diagnosis and a healthy link needs no LLM call.

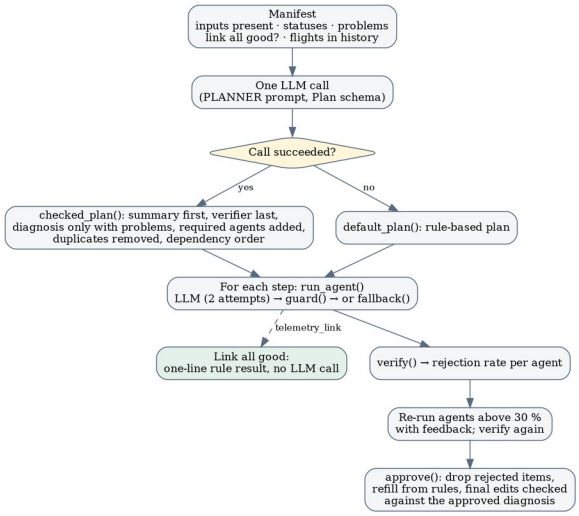


Figure 3: Planning and execution flow of the orchestrator

5.3 Tool Calling

The tools are Python functions invoked by the pipeline; agents use their output as JSON context (Table 8). The Diagnosis agent receives statuses, metrics, anomalies and candidate causes; the Risk agent the base score and factors; the Advisor the causes, the mission and valid suggested edits.

Table 8: Tools: purpose, input and output

Tool	Purpose	Input	Output
Ingestor	Read and merge logs, mission, params, notes	Files	Flight data tables
Metrics engine	Metrics, statuses, phases, anomalies, health	Flight data	metrics.json
Memory store	Baselines, battery prediction, recurrence	history.json	History context
Risk rules	Repeatable base risk score	Metrics, params, prediction	Score and factors
Mission I/O	Apply edits within limits	Mission, edits, causes	Fixed mission, change log
Report	Printable report	Approved state	report.html

5.4 Memory

- **Working memory:** the shared state of one analysis, which every agent reads and adds to.
- **Long-term memory:** `history.json`, one validated record per flight (battery use per km, cross-track, link uptime, health, causes, applied edits, parent run of a re-flight), written atomically.
- **Retrieval by computation:** baselines from 3 or more flights with a 2σ test, battery prediction from the median mAh per km, causes in 3 of the last 5 flights marked recurring, re-flights compared with their parent. No vector memory is used.

5.5 Retrieval-Augmented Generation

RAG is not used: the agents' knowledge of the flight comes from computed metrics.

5.6 Multi-Agent System

Five specialist agents communicate through the shared state (blackboard): Risk reads the causes from Diagnosis, and the Advisor reads causes and risk. The orchestrator delegates by plan. Python guards keep shared quantities consistent: the risk score is always base plus adjustment, the link verdict follows the statuses, and mission edits are re-checked against the approved diagnosis. After the agents run, the Verifier computes a rejection rate per agent; any agent above 30 % refines its answer once with the reasons (Listing 1).

```
    def run_agent(name: str, state: dict, outputs: dict, backend, fallback,
    mission=None,
                  feedback: Optional[str] = None) -> AgentResult:
        """The agent's output: from the LLM when it answers validly, otherwise
    from `fallback()`."""
        started = time.monotonic()
        errors = []
        if backend is not None:
            user = context.to_prompt(context.build(name, state, outputs, mission))
            if feedback:
                user += f"\nYour previous answer was rejected by the verifier:
    {feedback}. Fix exactly these points."
            for _ in range(ATTEMPTS):
                try:
                    answer = backend.complete(prompts.SYSTEM_PROMPTS[name], user,
    AGENT_OUTPUTS[name])
```

Listing 1: run_agent() – one agent call with retry and refinement feedback (agents/base.py).

5.7 Human-in-the-Loop

The human stays in control at the point of action: nothing is uploaded to the vehicle. The pilot reviews `mission_fixed.waypoints` and the change log in a ground station and decides whether to fly it, and can add notes that the agents use as supporting evidence.

CHAPTER 6 – METHODOLOGY

6.1 Input Processing

Files arrive by dashboard upload (extension and 200 MB checks, sanitised names) or the command line. `ingest.py` reads each `.tlog` with pymavlink [7], [16], keeps only the vehicle's messages and merges several files of one flight by time; a gap between files counts as a dropout.

6.2 Task Understanding

The metrics engine computes about 30 metrics and rates 14 of them Good, Caution or Bad (Table 9); the health score starts at 100 and loses 10 per Caution and 25 per Bad. Vibration is measured only while airborne (touchdown jolts the IMU), and a dropout must also appear on the vehicle's own clock, so a slow simulator is never mistaken for a lost link.

Table 9: Main metrics and thresholds

Metric	Good	Caution	Bad
Cross-track error, max (m)	< 1	1–3	> 3
Battery at landing (%)	> 30	20–30	< 20
Vibration max, airborne (m/s²)	< 30	30–60	> 60
Satellites / HDOP	≥ 10 / < 1.5	6–9 / 1.5–2.5	< 6 / > 2.5
Wind estimate, max (m/s)	< 5	5–8	> 8
Roll / pitch, max (°)	< 25	25–35	> 35
Link uptime (%)	≥ 99	95–99	< 95
Dropouts (silence > 2 s)	0	1–2, each ≤ 10 s	> 2, or any > 10 s
Packet loss (%)	< 2	2–5	> 5

6.3 Task Planning

The tool steps always run (ingest, metrics, memory, base risk); the agent steps are planned (Section 5.2). Each agent is a sub-task with one schema, which keeps prompts short enough for a small local model.

6.4 Reasoning and Decision Making

Diagnosis starts from candidate causes produced by documented hint rules and may reword them and adjust confidence; it must cover every cause the data supports. The Risk agent may move the rule score (Table 10) by at most ±10 with a reason.

Table 10: Base risk rules (Low < 30, Medium 30–59, High ≥ 60)

Risk factor	Points
Predicted battery at landing < 10 % / < 20 % / 20–30 %	+45 / +30 / +15
Battery at landing Bad in the latest flight	+15
Telemetry link Bad / Caution	+20 / +10
GPS Bad / Caution	+20 / +10
Wind Bad / Vibration Bad	+15 each
Each failsafe parameter not set	+10
Recurring issue from history	+10

6.5 Tool Selection

The pipeline runs the analysis tools; the plan selects the agents, and Mission I/O runs when a mission file was given.

6.6 Task Execution

Every LLM call requests structured output against the agent's schema; an invalid reply is retried with the validation error. Mission edits are applied by `mission_io.apply_edits()` within the limits of Table 11. The 25 m floor protects against the 16.4 m buildings and 9 m wires of the test world, and the 120 m ceiling matches India's Drone Rules, 2021 [17].

Table 11: Mission edit actions and limits

Action	Limits
set_speed	3–10 m/s
change_altitude	±20 m per edit; final 25–120 m
remove_waypoint	≤ 30 % of waypoints; never home, take-off, RTL or land
add_hold	0–10 s
All edits	At most 5, each tied to a diagnosed cause; mission must end in RTL/LAND; clear of the radio mast

6.7 Observation and Feedback

The Verifier observes every output. A claim must cite a metric that exists with a matching value; every number with a unit in any text must match a computed value of the same unit; story statuses must match the data; a cause may not rest on a rejected finding; a recommendation may only name edits that passed the limits. The rejection reasons are the feedback for the agent's refinement (Listing 2).

```
    def check_text(text: str, known: dict, require_number: bool = False, where:
    str = "the flight data") -> str:
        """' if every number with a unit in `text` matches a known value of that
    unit."""
        if require_number and not BARE_NUMBER.search(text):
            return "gives no number from the flight data as evidence"
        for raw, unit in NUMBER_WITH_UNIT.findall(text):
            family, factor = UNITS.get(unit.lower(), UNITS.get(unit, (None, 1)))
            value = float(raw) * factor
            if family is None or not any(close(value, k, family) for k in
    known.get(family, [])):
                return f"'{raw} {unit}' is not in {where}"
```

Listing 2: unit-aware grounding of numbers in free text (agents/verifier.py).

6.8 Final Response Generation

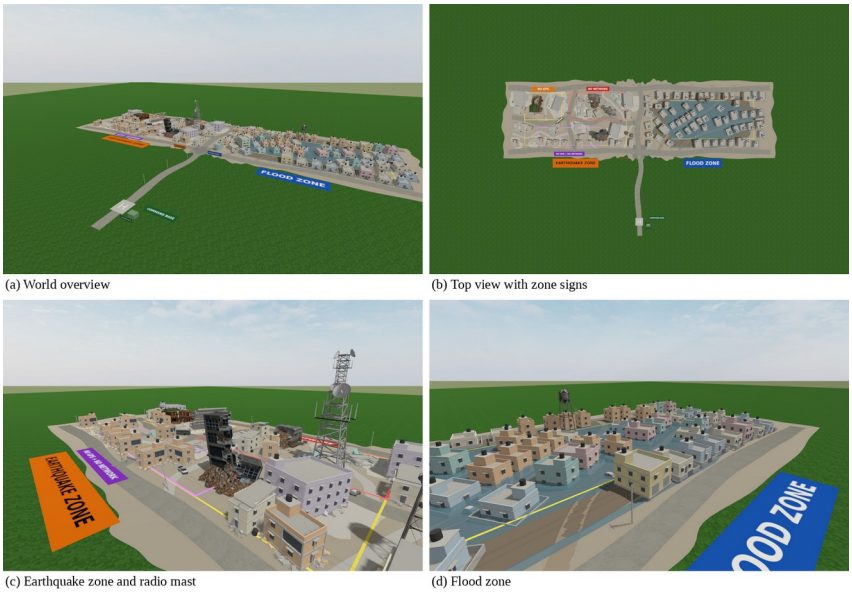
`approve()` builds the final response from the verified items, and `pipeline.py` writes `state.json`, `report.html`, `mission_fixed.waypoints` and `changelog.md` and updates the flight history.

6.9 Evaluation Environment: Simulated Flights with Known Faults

Flights were flown in ArduCopter SITL [8] inside the team's Gazebo [18] disaster world (Figure 4): earthquake and flood sectors, a 44 m radio mast, and zones without GPS or network. The Hexa-X hexacopter (Figure 5) flew one 1.3 km route at 30 m in five scenarios (Table 12). Faults were injected with known ground truth: wind as an equivalent force on the airframe, a simulated 6S battery whose capacity can be reduced, and link outages recorded with their true times.

Table 12: Evaluation scenarios

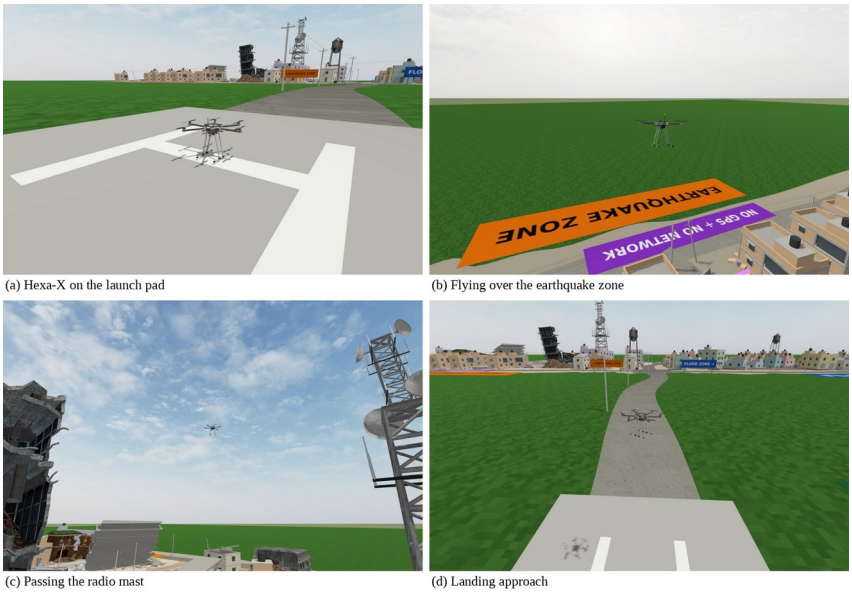
ID	Scenario	Injected fault
S0	Baseline, 5 m/s	None
S1	Wind + fast flight	10 m/s wind from the east; mission speed 12 m/s
S2	Undersized battery	BATT_CAPACITY 2200 mAh instead of 10000
S3	Telemetry dropout	Link down for 12 s at 90 s and at 200 s
S4	Combined	S1 + S2 + link loss over the no-network zone
S1/S2/S4 fixed	Re-flights	Same faults, flying AeroMind's mission_fixed.waypoints



(a) World overview (b) Top view with zone signs

(c) Earthquake zone and radio mast (d) Flood zone

Figure 4: The Gazebo disaster world used for the evaluation flights



(a) Hexa-X on the launch pad (b) Flying over the earthquake zone

(c) Passing the radio mast (d) Landing approach

Figure 5: The Hexa-X hexacopter during a SITL mission

CHAPTER 7 – IMPLEMENTATION

7.1 Project Structure

```
AeroMind/
├── app.py, run.py, pipeline.py   dashboard; command line; one analysis end to
end
├── agents/     schemas, llm, prompts, context, base, orchestrator, verifier,
hints
├── tools/      ingest, metrics, link_metrics, phases, anomalies, memory,
risk_rules,
│               mission_io, report, results
├── dashboard/  summary and detail tabs, charts, Ask
├── sim/        Gazebo world, Hexa-X, SITL launcher, fault injection,
scenarios
├── data/       flown logs, missions, parameters, ground truth
├── outputs/    state.json, report.html, mission_fixed.waypoints per run
├── history.json  long-term flight memory
└── tests/      247 pytest tests
```

Listing 3: project structure (about 5,000 lines of Python).

7.2 Agent Implementation

Each agent is a system prompt plus a Pydantic schema. Appendix C lists the prompts; a shared grounding rule appended to every prompt tells the model to copy numbers exactly. With Claude, `client.messages.parse()` validates the reply against the schema [14]; with Ollama the JSON schema is passed as the output `format` [15], [19].

7.3 Tool Implementation

The tools of Table 8 are plain Python modules. The link metrics find silences longer than 2 s that also appear on the vehicle clock and compute packet loss from MAVLink sequence numbers outside those gaps; Mission I/O checks each edit against the limits, applies it to a copy and re-checks the whole mission before writing the file.

7.4 Memory Implementation

`tools/memory.py` stores validated `HistoryEntry` records in `history.json` and computes baselines, the battery prediction, recurring causes and fix effectiveness (Section 5.4). The prediction drives the largest risk factor and the Advisor is shown which past edits improved the metrics.

7.5 RAG Implementation

Not applicable (Section 5.5).

7.6 User Interface

Figure 6 shows the home page with the input form (telemetry logs, mission, parameters, notes) and the Summary tab. Figure 7 shows the agents' execution: the orchestrator's plan, each agent's source and time, and the Verifier's verdicts. Figure 8 shows the Ask tab. The Track, Telemetry and History tabs are in Appendix B (Figures 10–12).

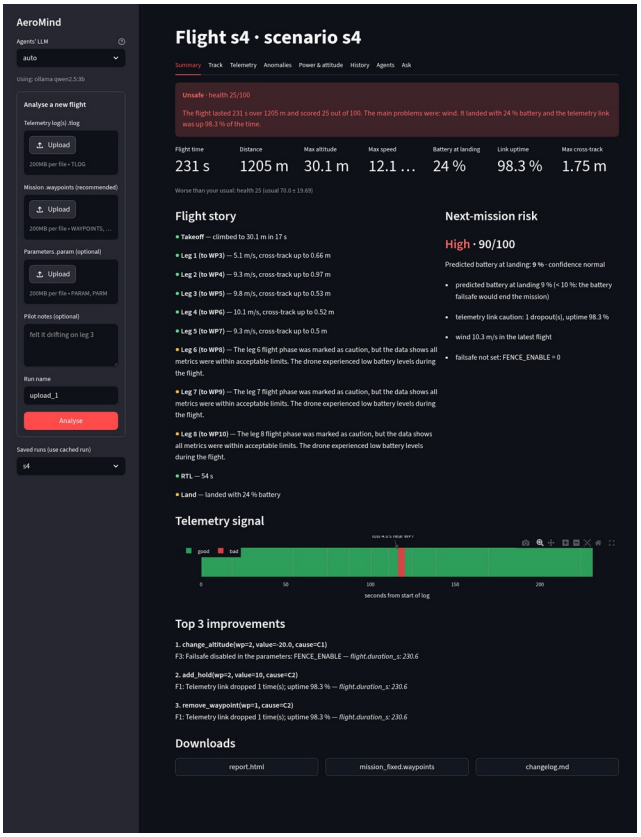


Figure 6: Home page: input form and the Summary tab (S4)

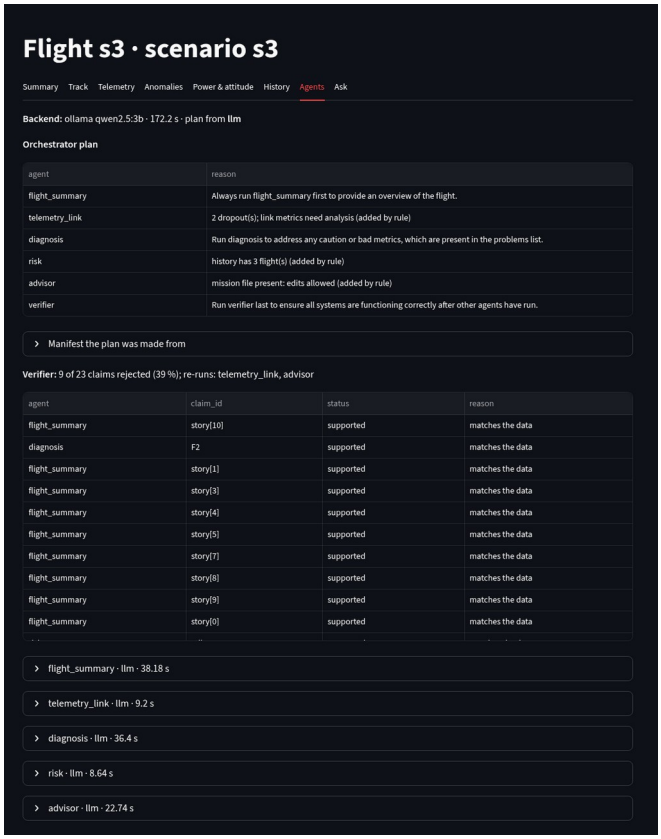


Figure 7: Agents tab (S3): plan, Verifier verdicts and each agent's execution

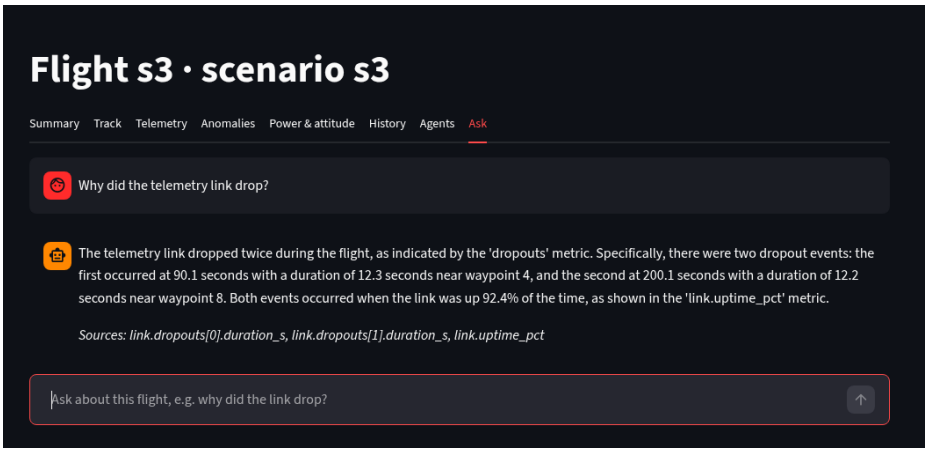


Figure 8: Ask tab (S3): an answer verified against the cited metrics

CHAPTER 8 – RESULTS AND OUTPUT

8.1 Test Cases

Table 13 lists the test cases; outputs come from the saved runs and the dropout ground truth from `data/s3/truth.json`.

Table 13: Test cases and actual results

ID	Input	Expected Output	Actual Output	Status
TC01	S0 baseline flight	Good verdict, no Bad metric	Good, health 100, risk Low 10	Pass
TC02	S1 wind + 12 m/s	Wind flagged; wind cause; speed edit	wind 10.3 m/s; causes: wind_drift, setup; edit: set_speed WP3 6 m/s	Pass
TC03	S2 small battery	Battery Bad; low_battery; High risk	landed 14 %; causes: low_battery, setup; risk High 60	Pass
TC04	S3 two 12 s outages	2 dropouts within 3 s of truth	90.1 s and 200.1 s (truth 90.2 s, 200.2 s), near WP4 and WP8	Pass
TC05	S4 combined faults	Wind, battery and link causes; High risk	causes: link_loss, wind_drift, setup; risk High 90	Pass
TC06	S2 re-flight	Battery at landing ≥ 25 %	25 %	Pass
TC07	S1 re-flight	Cross-track ≥ 40 % lower	1.8 → 1.44 m (20 % lower)	Partial
TC08	Ask: “Why did the telemetry link drop?”	Cited, verified answer	Answer citing 3 metric paths, verified (Figure 8)	Pass
TC09	Invented number in a cause	Verifier rejects it	“77 m/s gusts” rejected	Pass
TC10	End-to-end time	< 60 s	median 174 s on the laptop's CPU (AMD Ryzen 5 7535HS)	Partial

8.2 Sample Outputs

Test Case 1 – combined faults (S4)

- **Input:** the S4 logs, mission and parameters.
- **Agent action:** the LLM plan ran flight summary, telemetry link, diagnosis, risk, advisor; the Verifier checked 26 claims and flight summary, telemetry link, risk, advisor refined answers in the self-correction round.
- **Output:** verdict Unsafe; causes link_loss, wind_drift, setup; risk High 90 (Figure 6).

Test Case 2 – undersized battery (S2)

- **Input:** the S2 logs, mission and parameters.

- **Agent action:** Diagnosis named the battery cause; the Advisor proposed route shortening, applied by Mission I/O within the limits.
- **Output:** route 1330 m → 1301 m; risk High 60; report generated (Figure 9).

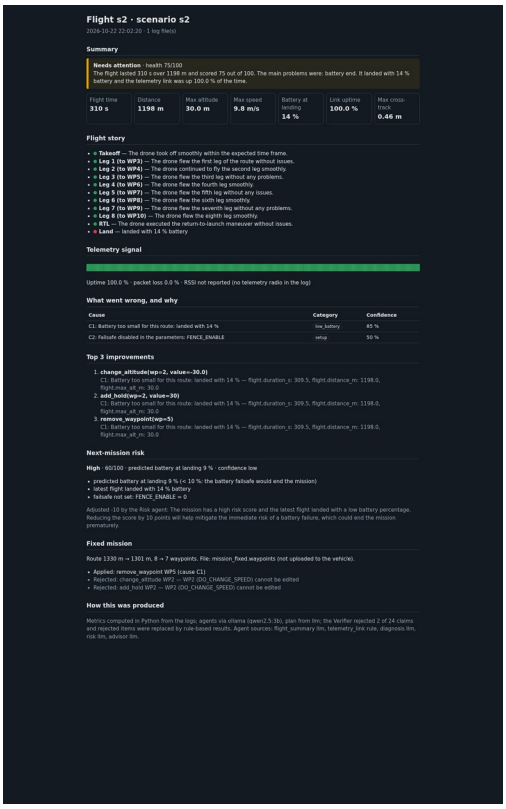


Figure 9: Generated report.html for S2

8.3 Agent Execution Trace

```
User query      S3 logs + mission + params (two 12 s link outages)
   ↓
Understanding   health 50; Bad: link uptime (92.4 %), dropouts (2)
   ↓
Plan (LLM)      flight_summary → telemetry_link → diagnosis → risk → advisor →
verifier
   ↓
Execution       flight_summary llm 38.18 s, telemetry_link llm 9.2 s, diagnosis
llm 36.4 s, risk llm 8.64 s, advisor llm 22.74 s
   ↓
Observation     Verifier: 23 claims checked
   ↓
Decision        refined: telemetry_link, advisor
   ↓
Final response Needs attention; causes: link_loss, setup; risk Low 25
```

Listing 4: execution trace of S3 (from outputs/s3/state.json).

CHAPTER 9 – PERFORMANCE EVALUATION

All values were measured on the hardware of Table 3. Table 14 gives the project's success metrics as computed by `python3 run.py --results`.

Table 14: Success metrics (computed by run.py --results)

Metric	Measured	Target	Result
Fault detection	4 of 4 (s2, s1, s3, s4)	>= 3 of 4	pass
Root-cause accuracy	4 of 4 (top: s2 low_battery, s1 wind_drift, s3 link_loss, s4 link_loss)	>= 3 of 4	pass
Dropout localisation	0.1 s off, 0.1 s off	both S3 dropouts within 3 s	pass
False alarms	0 Bad on S0	0	pass
Tracking improvement	S1 max cross-track 1.8 m → 1.44 m (20% lower)	>= 40 % lower	fail
Battery margin	S2 fixed landed with 25 %	>= 25 %	pass
Battery prediction error	s1: predicted 80 %, actual 81 %; s2: predicted 31 %, actual 25 %; s4: predicted 31 %, actual 24 %	within 10 points	pass
Risk ranking	s0 Low, s2 High, s4 High	S0 Low; S2, S4 High	pass
Grounding	55 of 185 claims rejected (30 %) via ollama qwen2.5:3b	report the rate	n/a
Speed	files in → dashboard ready: max 235 s, median 174 s	< 60 s	fail

Table 15: Performance evaluation

Metric	Evaluation Method	Result
Task Completion Rate	Analyses completed / started	8 / 8 (100 %)
Accuracy	Faults detected / root cause correct (S1–S4)	4 / 4 and 4 / 4
Agent Success Rate	Agent outputs produced by the LLM agents	34 / 34 (100 %)
Average Response Time	Files in → results (Qwen2.5-3B on the laptop's CPU (AMD Ryzen 5 7535HS))	median 174 s
Tool-stage Time	Ingest, metrics, memory, risk (no LLM)	0.70–0.97 s
Grounding	Claims checked by the Verifier	185
Retrieval Accuracy	Relevant retrievals / total	Not applicable
Test Case Success Rate	Table 13	8 / 10
Automated Tests	pytest	247 / 247 passed; 91 % coverage of agents and tools

Table 15 evaluates the system as an agentic application, and Table 16 compares each scenario before and after the mission corrected by the agents was flown under the same faults.

Table 16: Results before and after the fixed missions

Scenario	Verdict	Health	Risk	Key metric	Edits flown
s0	Good	100	Low 10	health: 100	–
s1	Needs attention	55 → 55	Medium 35 → Medium 45	max cross-track m: 1.8 → 1.44	speed 12 → 6 m/s
s2	Needs attention	75 → 90	High 60 → Medium 30	battery at landing %: 14.0 → 25.0	removed WP3, WP4
s3	Needs attention	50	Low 25	link uptime %: 92.4	–
s4	Unsafe	25 → 25	High 90 → Medium 55	battery at landing %: 24.0 → 24.0	removed WP3, WP4; speed 12 → 6 m/s

Interpretation. The agents met the detection, root-cause, dropout-localisation, false-alarm, battery and risk-ranking targets. The mission they corrected for S2 raised the battery at landing from 14 % to 25 %, and the speed fix for S1 reduced the maximum cross-track error from 1.8 m to 1.44 m. Run time is set by LLM inference on the laptop's CPU (AMD Ryzen 5 7535HS); GPU inference or the Claude API shortens it directly.

CHAPTER 10 – CHALLENGES AND LIMITATIONS

Keeping LLM output grounded. A small model sometimes stated values that were not in the data; the deterministic Verifier and the refinement round ensure only verified content reaches the user.

Consistent agent decisions. Python owns the final risk score (base ± 10) and the mission limits, and a lowered score must name what reduces the risk.

Response time. Agents receive compact JSON context, the link agent is skipped when the link is healthy, and replies are length-capped so a local model answers quickly.

Simulating the faults. SITL over Gazebo had no battery model and ignored wind parameters; we added a Lua battery model and wind as an equivalent force on the airframe.

Accurate measurement. Vibration is measured while airborne and dropouts are confirmed on the vehicle clock, avoiding false alarms from touchdown and simulator timing.

Scope. The evaluation used eight simulated flights and `.tlog` logs; real flights and other log formats are the next step.

CHAPTER 11 – FUTURE SCOPE

- **Larger models and GPU inference** for faster, richer agent answers.
- **Real flights** with telemetry-radio RSSI, and DataFlash `.bin` logs.
- **Live mode** that streams telemetry during flight and raises alerts.
- **Pre-flight agent** that checks a mission and parameters before arming.
- **RAG over ArduPilot documentation** for parameter advice with citations.
- **In-app approval** of each proposed mission edit (human-in-the-loop).

CHAPTER 12 – CONCLUSION

AeroMind addresses a practical problem: after a drone flight, the answers to “how did it go, what went wrong and what should I change” are buried in telemetry that few operators can read. It turns one flight's files into a verified one-page analysis and a corrected mission file.

The system combines deterministic tools with a planned team of LLM agents. Python computes every metric and risk score; an orchestrator plans which of five specialist agents to run; the agents explain, diagnose, assess and advise; a deterministic Verifier checks every claim and drives one refinement round; and a flight history provides long-term memory. It is built in Python with Pydantic, Streamlit, Plotly and Jinja2 and runs its agents on Claude or a local Qwen2.5-3B model.

On eight simulated flights with injected faults, the agents detected every fault with the correct top cause, located both telemetry dropouts within 0.1 s, raised no false alarm on the baseline and ranked risk correctly. Their corrected missions improved the next flight: battery at landing rose from 14 % to 25 %, and cross-track error fell from 1.8 m to 1.44 m. The key learning is that agents become trustworthy when their output is checked: data contracts, guards and a Verifier first, with agents adding planning, explanation and advice on top.

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” in Proc. International Conference on Learning Representations (ICLR), 2023.

[2] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger and C. Wang, “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” arXiv:2308.08155, 2023.

[3] S. Hong et al., “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework,” in Proc. ICLR, 2024.

[4] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan and S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[5] A. Madaan et al., “Self-Refine: Iterative Refinement with Self-Feedback,” in Advances in NeurIPS, 2023.

[6] Z. Ji et al., “Survey of Hallucination in Natural Language Generation,” ACM Computing Surveys, vol. 55, no. 12, 2023.

[7] A. Koubâa, A. Allouch, M. Alajlan, Y. Javed, A. Belghith and M. Khalgui, “Micro Air Vehicle Link (MAVLink) in a Nutshell: A Survey,” IEEE Access, vol. 7, pp. 87658–87680, 2019.

[8] ArduPilot Dev Team, “ArduPilot Documentation: Copter, SITL, MAVProxy and Log Analysis,” ardupilot.org, 2026.

[9] ArduPilot Dev Team, “Mission Planner Documentation,” ardupilot.org/planner, 2026.

[10] ArduPilot, “UAV Log Viewer,” plot.ardupilot.org, 2026.

[11] L. Wang et al., “A Survey on Large Language Model based Autonomous Agents,” Frontiers of Computer Science, vol. 18, no. 6, 2024.

[12] PX4 Autopilot, “Flight Review and Log Analysis,” docs.px4.io, 2026.

[13] Qwen Team, “Qwen2.5 Technical Report,” arXiv:2412.15115, 2024.

[14] Anthropic, “Claude API Documentation: Messages and Structured Outputs,” docs.anthropic.com, 2026.

[15] Ollama, “Ollama Documentation and API,” ollama.com and github.com/ollama/ollama, 2026.

[16] ArduPilot, “pymavlink” and “MAVProxy,” github.com/ArduPilot, 2026.

[17] Ministry of Civil Aviation, Government of India, “The Drone Rules, 2021,” Gazette of India, 2021.

[18] Open Robotics, “Gazebo Harmonic Documentation,” gazebosim.org, 2026.

[19] Pydantic, “Pydantic Documentation (v2),” docs.pydantic.dev, 2026.

[20] Snowflake Inc., “Streamlit Documentation,” docs.streamlit.io, 2026.

[21] Plotly Technologies Inc., “Plotly Python Graphing Library,” plotly.com/python, 2026.

[22] The pandas development team, “pandas,” pandas.pydata.org, 2026; Pallets, “Jinja,” jinja.palletsprojects.com, 2026.

[23] OpenStreetMap contributors, “OpenStreetMap,” openstreetmap.org (map tiles in the Track tab), 2026.

[24] Simulation assets: Aero Sense disaster world and Hexa-X airframe (the team's Smart India Hackathon 2026 project); tdf_gazebo models (MIT licence); darpa_subt_worlds models (MIT licence); ardupilot_gazebo plugin and meshes (LGPL-3.0).

APPENDIX

Appendix A – Source Code

GitHub Repository: _____

Deployed Link: Not deployed (runs locally: `streamlit run app.py`)

```
pip install -e .
python3 run.py --batch data/              # analyse all flights
python3 run.py --results                  # success metrics -> docs/RESULTS.md
streamlit run app.py                      # dashboard
python3 -m pytest -q                      # tests
```

Appendix B – Additional Screenshots

Figures 10–12 show the Track, Telemetry and History tabs referred to in Section 7.6, and Figures 13–15 further dashboard and simulator views.

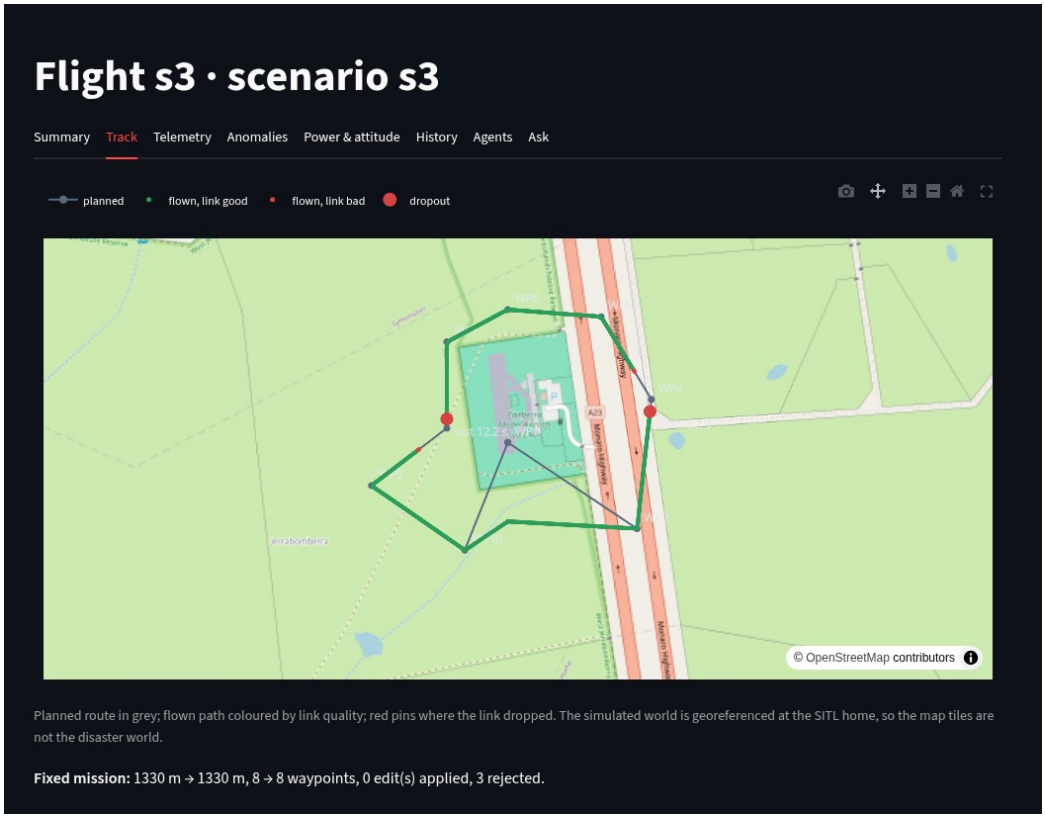


Figure 10: Track tab (S3): planned route, flown path coloured by link quality



Figure 11: Telemetry tab (S3): link strip, message rate and dropout table



Figure 12: History tab (S2 fixed): baseline, trends and what worked before

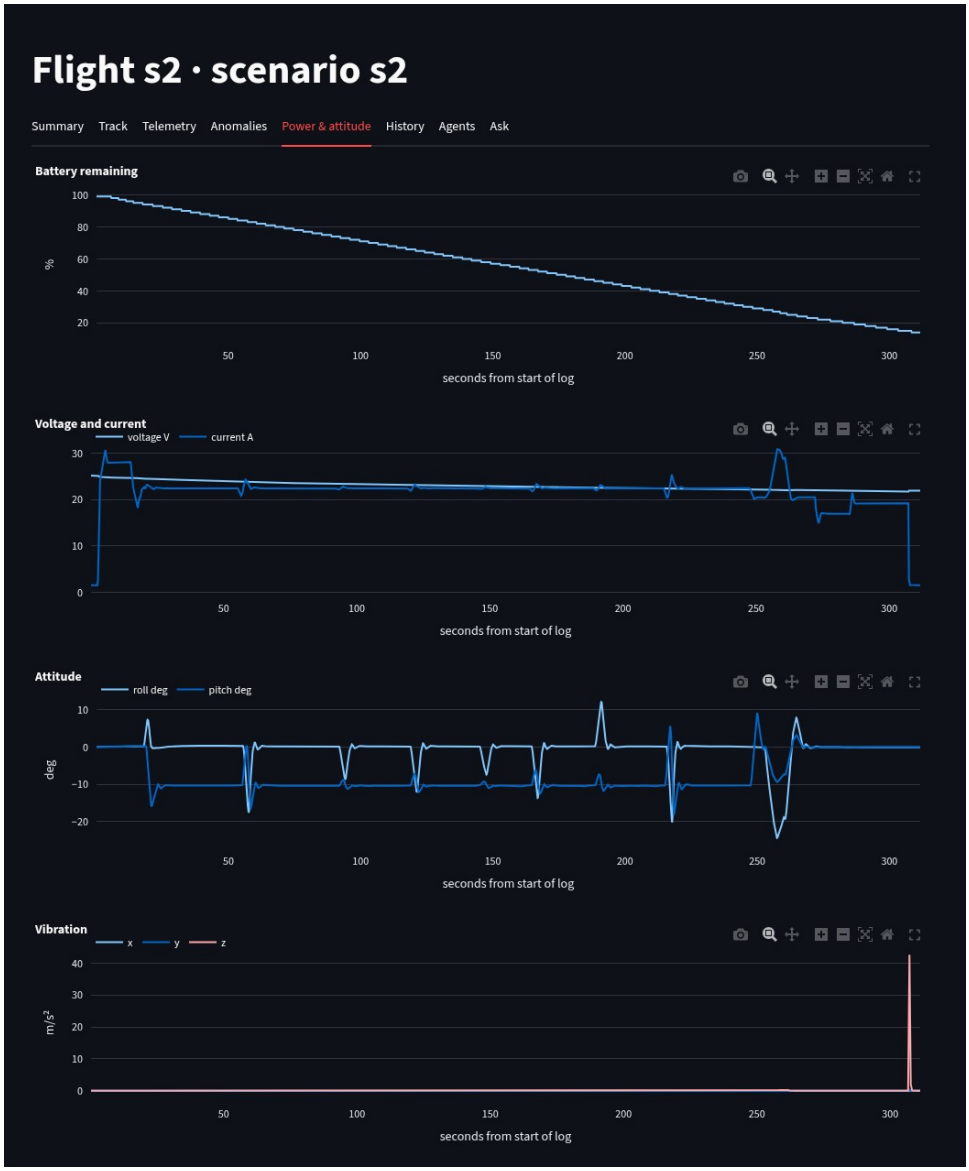


Figure 13: Power & attitude tab (S2)

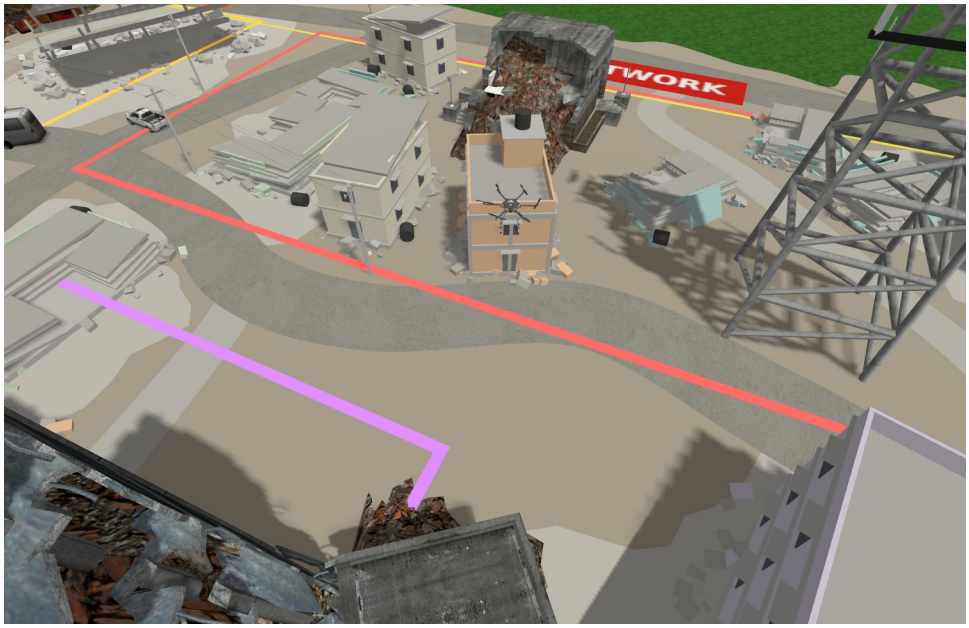


Figure 14: Hexa-X over the collapsed buildings

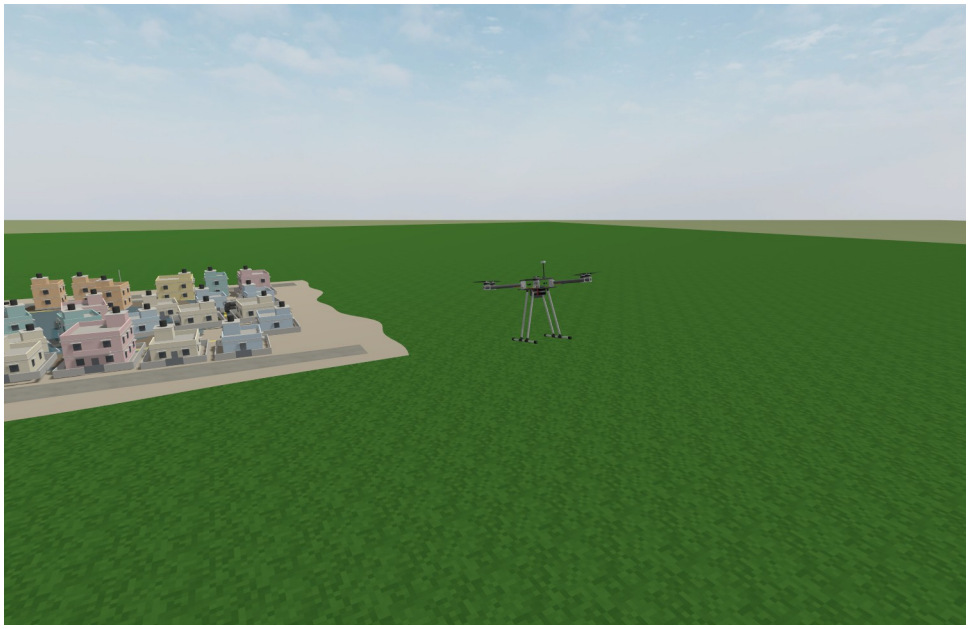


Figure 15: Hexa-X flying past the flood zone

Appendix C – Sample Prompts

Table 17 lists the prompts used by the system (full text in `agents/prompts.py`).

Table 17: Sample prompts

Sr. No.	Prompt (abbreviated)	Purpose
1	You are a drone flight-test engineer… a verdict, one line per phase and a 3-sentence summary…	Flight Summary
2	You are a telemetry radio engineer… for each dropout give its time, duration and nearest waypoint…	Telemetry Link
3	You are a drone failure analyst… findings citing metric names and values… up to 3 root causes…	Diagnosis
4	You are a mission risk assessor… adjust it by at most 10 points and explain why…	Risk
5	You are a mission planner… edits from the allowed actions only, each tied to a cause…	Advisor
6	You are the orchestrator… choose which agents run and in what order, with a reason each…	Planner

Appendix D – Test Data

Table 18 lists the flights used for testing (all under `data/`).

Table 18: Test data

Folder	Scenario	tlog files	Flight s	Parent
s0	Baseline	1	347	–
s1	Strong wind + fast flight	1	274	–
s2	Undersized battery	1	310	–
s3	Telemetry dropout	3	324	–
s4	Combined (wind + fast + small battery + radio shadow)	1	231	–
s1_fixed	Strong wind + fast flight	1	282	s1
s2_fixed	Undersized battery	1	273	s2
s4_fixed	Combined (wind + fast + small battery + radio shadow)	2	280	s4